Министерство науки и высшего образования РФ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Ижевский государственный технический университет
имени М.Т. Калашникова»
(ИжГТУ имени М.Т. Калашникова)
Факультет «Математика и естественные науки»
Кафедра «Прикладная математика и информационные технологии»

Лабораторная работа № 1
по курсу **«Интеллектуальные системы в процессах управления»**
Вариант 7

Направление подготовки 01.03.02 «Прикладная математика»
Профиль – «Прикладной анализ данных и искусственный интеллект»

Проверил: К. И. Дизендорф
к. ф.-м. н., доцент

Выполнил: М. А. Панин
студент группы Б23-183-1

Ижевск - 2026

Содержание

Цель работы . . . 2
Постановка задачи . . . 2
- Входные данные (сенсоры) . . . 2
- Управляемые устройства . . . 2

Формализованная база знаний . . . 3
- Лингвистические переменные . . . 3
- Продукционные правила . . . 4

Алгоритм логического вывода . . . 4
Структура данных . . . 4
- Типы и перечисления (`data_structures.py`) . . . 4
- Состояния устройств (`states.py`) . . . 5

Контроллеры устройств . . . 6
- Правила управления устройствами . . . 7
- Основное освещение (`LightMainControl`) . . . 8
- Климат-контроль (`ClimateControl`) . . . 9
- Вентиляция (`FanControl`) . . . 10
- Система оповещения (`AlertControl`) . . . 10

Архитектура системы . . . 11
- Протокол контроллера . . . 11
- Оркестратор `ControlSystem` . . . 11
- Точка входа . . . 12

Тестирование системы . . . 13
- Входные конфигурации . . . 13
- Результаты тестирования . . . 13
- Анализ результатов . . . 14

Заключение . . . 14

Цель работы

Целью лабораторной работы является разработка прототипа экспертной системы управления микроклиматом «Умной комнаты» на языке программирования Python. Система должна принимать показания сенсоров (температура, влажность, уровень CO_2, освещённость, наличие людей и время суток) и на основе набора продукционных правил автоматически управлять устройствами помещения: климат-контролем, увлажнителем, вентиляцией, освещением и системой оповещения.

Постановка задачи

Требуется спроектировать и реализовать программный прототип, который:

1. Принимает на вход конфигурацию помещения (показания сенсоров).
2. Для каждого управляемого устройства вычисляет новое состояние по заранее заданным правилам.
3. Выводит полное состояние всех устройств после обработки.

Входные данные (сенсоры)

Таблица 1 - Описание входных параметров системы

Параметр	Тип	Единица	Описание
`temperature`	`int`	°C	Температура воздуха
`humidity`	`float`	%	Относительная влажность
`co2`	`int`	ppm	Концентрация CO_2
`luminosity`	`int`	лк	Освещённость
`people`	`bool`	-	Наличие людей в комнате
`time_of_day`	`Time`	-	Время суток: утро / день / вечер / ночь

Управляемые устройства

Таблица 2 - Управляемые устройства системы

Устройство	Функция
Климат-контроль	Нагрев или охлаждение воздуха
Увлажнитель	Повышение влажности при низких значениях
Вентиляция	Управление скоростью вентилятора при высоком CO_2
Основное освещение	Регулировка яркости в зависимости от условий
Ночное освещение	Мягкая подсветка ночью при наличии людей

Система оповещения	Предупреждение или тревога при критических значениях

Формализованная база знаний

Лингвистические переменные

Система оперирует шестью входными и шестью выходными лингвистическими переменными.

Таблица 3 - Входные лингвистические переменные

Переменная	Диапазон	Термы (нечёткие значения)
`temperature`	−20…50 °C	низкая (< 18 °C) · комфортная (18–26 °C) · тёплая (26–30 °C) · опасная (> 30 °C)
`humidity`	0…100 %	сухо (< 30 %) · норма (30–40 %) · влажно (≥ 40 %)
`co2`	0…5000 ppm	норма (≤ 1000 ppm) · повышенный (1000–1500 ppm) · высокий (1500–2000 ppm) · критический (> 2000 ppm)
`luminosity`	0…10000 лк	темно (< 50 лк) · слабо (50–100 лк) · тускло (100–200 лк) · умеренно (200–400 лк) · ярко (≥ 400 лк)
`people`	{Да, Нет}	люди есть · людей нет
`time_of_day`	-	утро · день · вечер · ночь

Таблица 4 - Выходные лингвистические переменные

Переменная	Устройство	Термы
`COND_HEAT`	Климат-контроль	выкл. · нагрев_слабый · нагрев_средний · охлаждение_слабое · охлаждение_сильное
`HUMID`	Увлажнитель	выкл. · слабый · сильный
`FAN`	Вентиляция	выкл. · средняя_скорость · высокая_скорость
`LIGHT_NIGHT`	Ночное освещение	выкл. · вкл.
`LIGHT_MAIN`	Основное освещение	выкл. · слабая · средняя · высокая · аварийная (мигание)
`ALERT`	Система оповещения	тишина · предупреждение · тревога

Продукционные правила

Полный набор продукционных правил приведён в разделе «Контроллеры устройств». Правила формируют 19 условий, покрывающих все штатные и аварийные режимы работы системы. Конфликты при пересечении условий (R3/R4, R7/R6, R13/R14, R16/R19) разрешены явными взаимоисключающими паттернами в конструкции `match`/`case`.

Алгоритм логического вывода

Система реализует прямой цепной вывод (forward chaining) без нечёткой логики: условия правил проверяются точно по числовым порогам. Цикл вывода для каждого вызова `ControlSystem.run(config)` состоит из следующих шагов:

1. **Получение конфигурации.** Оркестратор принимает объект `SystemConfig` с текущими показаниями сенсоров.
2. **Сброс состояний.** В начале `compute(config)` каждый контроллер пересоздаёт свой объект состояния (`self.state = ...States()`). Это гарантирует, что устаревшие значения не переносятся в следующий цикл («залипание» исключено).
3. **Вычисление состояний.** Каждый контроллер независимо вычисляет своё состояние с помощью `match`/`case`. Паттерны проверяются сверху вниз; срабатывает первый подходящий - срабатывание остальных не происходит.
4. **Разрешение конфликтов.** Конкурирующие правила реализованы взаимоисключающими паттернами, а не порядком `elif`. Например, R4 задан как `(False, False, True, True, _)` - второй элемент `False` явно гарантирует T ≤ 30 °C, исключая область R3.
5. **Вывод состояний.** Для каждого устройства печатается текущее состояние и код сработавшего правила.
6. **Отображение diff.** `ControlSystem` сохраняет снимок состояния (`dataclasses.asdict`) после каждого `run()` и при следующем вызове выводит только изменившиеся поля.

Структура данных

Типы и перечисления (`data_structures.py`)

Модуль `data_structures.py` определяет типы, перечисления и класс конфигурации системы. Для определения типов используется `NewType` вместо

`TypeAlias`: это позволяет статическому анализатору различать `Temperature`, `Ppm`, `Lux` и `Percent` как несовместимые типы, а не просто синонимы `int`/`float`.

Листинг 1 - `data_structures.py` - типы и перечисления

```
1 from dataclasses import dataclass
2 from enum import Enum
3 from typing import NewType
4
5 Ppm         = NewType("Ppm", int)
6 Lux         = NewType("Lux", int)
7 Percent     = NewType("Percent", float)
8 Temperature = NewType("Temperature", int)
9
10 class Mode(Enum):
11     OFF    = "off"
12     LOW    = "low"
13     MEDIUM = "medium"
14     HIGH   = "high"
15
16 class Time(Enum):
17     MORNING   = "morning"
18     AFTERNOON = "afternoon"
19     EVENING   = "evening"
20     NIGHT     = "night"
21
22 class Alert(Enum):
23     SILENT  = "silent"
24     WARNING = "warning"
25     ALARM   = "alarm"
```

Класс `SystemConfig` объединяет все входные параметры в единую структуру данных:

Листинг 2 - `data_structures.py` - класс конфигурации системы

```
1 @dataclass
2 class SystemConfig:
3     temperature: Temperature
4     humidity:    Percent
5     co2:         Ppm
6     luminosity:  Lux
7     people:      bool
8     time_of_day: Time
```

Состояния устройств (`states.py`)

Каждое устройство имеет собственный датакласс состояния. Все поля инициализируются значениями по умолчанию (устройство выключено). Поле `rule`

хранит код последнего сработавшего правила и используется при выводе состояния и отслеживании изменений.

Листинг 3 - `states.py` - датаклассы состояний устройств

```
1  @dataclass
2  class ClimateControlStates:
3      is_working: bool       = False
4      heating:    Mode       = Mode.OFF
5      cooling:    Mode       = Mode.OFF
6      rule:       str | None = None
7
8  @dataclass
9  class HumidityControlStates:
10     is_working:  bool       = False
11     humidifying: Mode       = Mode.OFF
12     rule:        str | None = None
13
14 @dataclass
15 class FanControlStates:
16     is_working: bool       = False
17     speed:      Mode       = Mode.OFF
18     rule:       str | None = None
19
20 @dataclass
21 class LightMainStates:
22     is_working: bool       = False
23     brightness: Mode       = Mode.OFF
24     rule:       str | None = None
25
26 @dataclass
27 class LightNightStates:
28     is_working: bool       = False
29     rule:       str | None = None
30
31 @dataclass
32 class AlertStates:
33     is_working: bool       = False
34     alert:      Alert      = Alert.SILENT
35     rule:       str | None = None
```

Контроллеры устройств

Каждый контроллер реализует два метода:

- `compute(config)` - принимает конфигурацию, сбрасывает состояние до значений по умолчанию и вычисляет новое состояние по правилам с помощью `match/case`;
- `__str__()` - возвращает строковое представление текущего состояния.

Сброс состояния в начале каждого `compute()` устраняет «залипание» данных между запусками: если условие срабатывало в прошлом цикле, но уже не выполняется, устройство корректно переходит в выключенное состояние.

Правила управления устройствами

Таблица 5 - Продукционные правила контроллеров

№	Устройство	Условие	Действие
R1	Климат-контроль	$T < 18$ °C ∧ ночь	Нагрев: low
R2	Климат-контроль	$T < 18$ °C ∧ не ночь	Нагрев: medium
R3	Климат-контроль	$T > 30$ °C	Охлаждение: high
R4	Климат-контроль	26 °C $< T \leq 30$ °C ∧ люди	Охлаждение: low
R5	Климат-контроль	иначе (18 °C $\leq T \leq 26$ °C)	Выключен
R6	Вентиляция	$CO_2 > 1000$ ppm ∧ люди	Скорость: medium
R7	Вентиляция	$CO_2 > 1500$ ppm	Скорость: high
R8	Увлажнитель	$H < 30$ %	Увлажнение: high
R9	Увлажнитель	30 % $\leq H < 40$ %	Увлажнение: low
R10	Увлажнитель	$H \geq 40$ %	Выключен
R11	Ночн. освещение	ночь ∧ люди ∧ $Lux < 50$	Включить
R12	Ночн. освещение	иначе	Выключить
R13	Оповещение	$T > 35$ °C ∨ $CO_2 > 2000$ ppm	Тревога: alarm
R14	Оповещение	($T > 32$ °C ∨ $CO_2 > 1800$ ppm) ∧ ¬R13	Предупреждение: warning
R15	Оповещение	иначе	Тишина: silent
R16	Осн. освещение	ночь ∧ люди ∧ $Lux < 100$	Яркость: high
R17	Осн. освещение	люди ∧ $Lux < 200$	Яркость: medium
R18	Осн. освещение	люди ∧ $Lux < 400$	Яркость: low
R19	Осн. освещение	$T > 35$ °C ∨ $CO_2 > 2000$ ppm (∧ ¬R16–R18)	Аварийный: high + мигание

В базе правил имеются пересечения условий. Они разрешены явным указанием приоритета непосредственно в паттернах `match/case`:

- **R3 vs R4:** при T > 30 °C условие R4 (T > 26 °C ∧ люди) тоже выполняется. R4 переопределён как 26 °C < T ≤ 30 °C, что в коде выражено паттерном `case (False, False, True, True, _)` - второй элемент кортежа явно равен `False` (T не превышает 30 °C).
- **R7 vs R6:** при CO_2 > 1500 ppm и наличии людей применимы оба правила. R7 проверяется первым; R6 задан паттерном `case (False, True, True)`, явно требующим $CO_2 \leq 1500$ ppm.
- **R13 vs R14:** условие R13 является подмножеством условия R14. R14 активируется только при ложном условии тревоги: `case (False, True)`.
- **R19 vs R16–R18:** аварийный режим (R19) проверяется первым в кортеже `(alarm, people, ...)` и перекрывает все штатные правила освещения вне зависимости от наличия людей.

Основное освещение (`LightMainControl`)

Листинг 4 - `controllers.py` - контроллер основного освещения

```
1  class LightMainControl:
2      def compute(self, config: SystemConfig):
3          self.state    = LightMainStates()
4          people        = config.people
5          time_of_day   = config.time_of_day
6          luminosity    = config.luminosity
7          # Аварийное условие совпадает с R13 (AlertControl)
8          alarm = (
9              config.temperature > Temperature(35)
10             or config.co2 > Ppm(2000)
11         )
12
13         match (
14             alarm, people, time_of_day,
15             luminosity < Lux(100),
16             luminosity < Lux(200),
17             luminosity < Lux(400),
18         ):
19             case (True, _, _, _, _, _):          # R19: авария
   - мигание, max яркость
20                 self.state.is_working = True
21                 self.state.brightness = Mode.HIGH
22                 self.state.flashing   = True
23                 self.state.rule       = "R19"
24             case (False, False, _, _, _, _):     # нет людей,
   нет аварии
25                 pass
26             case (False, True, Time.NIGHT, True, _, _):  # R16
27                 self.state.is_working = True
```

```
                self.state.brightness = Mode.HIGH
                self.state.rule       = "R16"
            case (False, True, _, _, True, _):   # R17
                self.state.is_working = True
                self.state.brightness = Mode.MEDIUM
                self.state.rule       = "R17"
            case (False, True, _, _, _, True):   # R18
                self.state.is_working = True
                self.state.brightness = Mode.LOW
                self.state.rule       = "R18"
            case _:
                pass
```

Климат-контроль (`ClimateControl`)

Листинг 5 - `controllers.py` - контроллер климата

```
class ClimateControl:
    def compute(self, config: SystemConfig):
        self.state  = ClimateControlStates()
        t           = config.temperature
        people      = config.people
        time_of_day = config.time_of_day

        match (
            t < Temperature(18),
            t > Temperature(30),
            t > Temperature(26),
            people,
            time_of_day,
        ):
            case (True, _, _, _, Time.NIGHT):   # R1
                self.state.is_working = True
                self.state.heating    = Mode.LOW
                self.state.rule       = "R1"
            case (True, _, _, _, _):            # R2
                self.state.is_working = True
                self.state.heating    = Mode.MEDIUM
                self.state.rule       = "R2"
            case (_, True, _, _, _):            # R3
                self.state.is_working = True
                self.state.cooling    = Mode.HIGH
                self.state.rule       = "R3"
            case (False, False, True, True, _): # R4: явно T <= 30
                self.state.is_working = True
                self.state.cooling    = Mode.LOW
                self.state.rule       = "R4"
            case _:                             # R5
                self.state.rule       = "R5"
```

Вентиляция (`FanControl`)

Листинг 6 - `controllers.py` - контроллер вентиляции

```
class FanControl:
    def compute(self, config: SystemConfig):
        self.state = FanControlStates()
        co2        = config.co2
        people     = config.people

        match (co2 > Ppm(1500), co2 > Ppm(1000), people):
            case (True, _, _):        # R7: приоритет над R6
                self.state.is_working = True
                self.state.speed      = Mode.HIGH
                self.state.rule       = "R7"
            case (False, True, True): # R6: явно CO2 <= 1500
                self.state.is_working = True
                self.state.speed      = Mode.MEDIUM
                self.state.rule       = "R6"
            case _:
                pass
```

Система оповещения (`AlertControl`)

Листинг 7 - `controllers.py` - контроллер оповещения

```
class AlertControl:
    def compute(self, config: SystemConfig):
        self.state  = AlertStates()
        temperature = config.temperature
        co2         = config.co2

        alarm   = temperature > Temperature(35) or co2 > Ppm(2000)
        warning = temperature > Temperature(32) or co2 > Ppm(1800)

        match (alarm, warning):
            case (True, _):     # R13
                self.state.is_working = True
                self.state.alert      = Alert.ALARM
                self.state.rule       = "R13"
            case (False, True): # R14: явно не тревога
                self.state.is_working = True
                self.state.alert      = Alert.WARNING
                self.state.rule       = "R14"
            case _:             # R15
                self.state.rule       = "R15"
```

Архитектура системы

Протокол контроллера

Для обеспечения единообразного интерфейса всех устройств в модуле `main_controller.py` определён структурный подтип (`Protocol`):

Листинг 8 - `main_controller.py` - протокол `Controller`

```
from typing import Any, Protocol
from data_structures import SystemConfig

class Controller(Protocol):
    state: Any

    def compute(self, config: SystemConfig) -> None: ...
    def __str__(self) -> str: ...
```

Использование `Protocol` вместо абстрактного базового класса позволяет применять структурную типизацию: любой класс, реализующий атрибут `state` и методы `compute` и `__str__`, автоматически считается совместимым, без явного наследования. Атрибут `state` объявлен как `Any`, поскольку у каждого контроллера он имеет собственный тип датакласса.

Оркестратор `ControlSystem`

Класс `ControlSystem` вызывает `compute` для каждого устройства, выводит его состояние и отображает изменения относительно предыдущего запуска:

Листинг 9 - `main_controller.py` - класс `ControlSystem`

```
class ControlSystem:
    def __init__(self, devices: Iterable[Controller]):
        self.devices      = list(devices)
        self._prev_states = [None] * len(self.devices)

    def run(self, config: SystemConfig) -> None:
        for i, device in enumerate(self.devices):
            prev = self._prev_states[i]
            device.compute(config)
            curr = asdict(device.state)

            print(device)
            if prev is not None:
                changes = {
                    k: (prev[k], curr[k])
                    for k in curr if prev[k] != curr[k]
                }
```

```
                for field, (old, new) in changes.items():
                    label = FIELD_NAMES.get(field, field)
                    print(f"  ↳ {label}: {_fmt(old)} → {_fmt(new)}")

            self._prev_states[i] = curr
            print("-" * 40)
```

После каждого вывода состояния устройства `ControlSystem` сравнивает текущий снимок состояния (`dataclasses.asdict`) с предыдущим и печатает только изменившиеся поля. При первом запуске diff не выводится.

Точка входа

Листинг 10 - `main.py` - инициализация системы и запуск симуляции

```
controller = ControlSystem(
    devices=[
        ClimateControl(),
        HumidityControl(),
        FanControl(),
        LightMainControl(),
        LightNightControl(),
        AlertControl(),
    ]
)

configs = [
    SystemConfig(
        temperature=Temperature(15), humidity=Percent(50),
        co2=Ppm(1900), luminosity=Lux(400),
        people=True, time_of_day=Time.EVENING,
    ),
    SystemConfig(
        temperature=Temperature(12), humidity=Percent(50),
        co2=Ppm(2100), luminosity=Lux(100),
        people=True, time_of_day=Time.EVENING,
    ),
    SystemConfig(
        temperature=Temperature(15), humidity=Percent(25),
        co2=Ppm(1900), luminosity=Lux(110),
        people=True, time_of_day=Time.EVENING,
    ),
]

for i, config in enumerate(configs):
    controller.run(config)
    if i < len(configs) - 1:
        print("==" * 20 + "Новое состояние" + "==" * 20)
```

Тестирование системы

Для проверки работоспособности системы были проведены три теста с различными конфигурациями сенсоров.

Входные конфигурации

Таблица 6 - Входные параметры тестовых сценариев

Параметр	Ед.	Сцен. 1	Сцен. 2	Сцен. 3
Температура	°C	15	12	15
Влажность	%	50	50	25
CO_2	ppm	1900	2100	1900
Освещённость	лк	400	100	110
Люди	-	да	да	да
Время суток	-	вечер	вечер	вечер

Результаты тестирования

Таблица 7 - Состояния устройств по результатам тестирования

Устройство / параметр	Сценарий 1	Сценарий 2	Сценарий 3
Климат-контроль: включён	ДА	ДА	ДА
Климат-контроль: нагрев	medium	medium	medium
Климат-контроль: охлаждение	off	off	off
Климат-контроль: правило	R2	R2	R2
Увлажнитель: включён	НЕТ	НЕТ	ДА
Увлажнитель: режим	off	off	high
Увлажнитель: правило	R10	R10	R8
Вентиляция: включена	ДА	ДА	ДА
Вентиляция: скорость	high	high	high
Вентиляция: правило	R7	R7	R7
Осн. освещение: включено	НЕТ	ДА	ДА
Осн. освещение: яркость	off	**high + мигание**	medium
Осн. освещение: правило	-	**R19**	R17
Ночное освещение: включено	НЕТ	НЕТ	НЕТ
Ночное освещение: правило	R12	R12	R12
Оповещение: включено	ДА	ДА	ДА
Оповещение: режим	warning	**alarm**	warning
Оповещение: правило	R14	**R13**	R14

Анализ результатов

Сценарий 1. Температура 15 °C < 18 °C - климат-контроль включён на нагрев (R2). CO_2 = 1900 ppm > 1500 ppm - вентиляция на высокой скорости (R7). CO_2 также превышает порог 1800 ppm, но ниже 2000 ppm - система оповещения выдаёт предупреждение (R14). Влажность 50 % ≥ 40 % - увлажнитель выключен (R10). Освещённость 400 лк достаточна - основное освещение не включается.

Сценарий 2. CO_2 = 2100 ppm > 2000 ppm - система переходит в режим тревоги (R13, изменение: R14 → R13). Одновременно срабатывает аварийный режим основного освещения (R19): яркость переключается на максимальную с миганием вне зависимости от освещённости и наличия людей (изменение: выключено → high + мигание). Температура 12 °C < 18 °C - климат-контроль продолжает нагрев (R2).

Сценарий 3. Влажность 25 % < 30 % - увлажнитель включается в режиме `high` (R8, изменение: R10 → R8). CO_2 = 1900 ppm - система оповещения возвращается к предупреждению (R14, изменение: R13 → R14). Освещённость 110 лк < 200 лк - основное освещение продолжает работать на средней яркости (R17).

Заключение

В ходе лабораторной работы был разработан прототип экспертной системы управления микроклиматом «Умной комнаты» на языке Python. Система реализована в виде набора независимых контроллеров, каждый из которых инкапсулирует продукционные правила для одного устройства. Оркестратор `ControlSystem` координирует все контроллеры через структурный протокол `Controller`, что обеспечивает расширяемость системы без изменения существующего кода.

В процессе реализации были устранены следующие проблемы:

- **Залипание состояния** - каждый `compute(config)` начинается со сброса состояния до значений по умолчанию, что гарантирует корректный переход устройства в выключенное состояние при несрабатывании правил.
- **Пересечения в базе правил** - конфликты R3/R4, R7/R6, R13/R14 и R19/R16–R18 разрешены явным указанием взаимоисключающих паттернов в `match/case`, а не порядком `elif`.
- **Аварийный режим освещения** - правило R19 перехватывает управление основным освещением при срабатывании тревоги (ALERT = alarm): устанав-

ливает максимальную яркость и режим мигания вне зависимости от наличия людей или уровня освещённости.

- **Типобезопасность** - `NewType` вместо `TypeAlias` позволяет статическому анализатору различать `Temperature`, `Ppm`, `Lux` и `Percent`.
- **Отслеживание изменений** - `ControlSystem` сохраняет снимок состояния после каждого `run()` и выводит diff при следующем вызове, показывая только изменившиеся поля.

Тестирование на трёх сценариях подтвердило корректность срабатывания правил и работу механизма diff: система правильно отображает переходы между правилами (например, R14 → R13 при росте CO_2 выше 2000 ppm и обратно).

В качестве возможных направлений развития можно выделить:

- вынесение пороговых значений в конфигурационный файл;
- добавление модульных тестов на базе `pytest`;
- интеграцию с реальными сенсорами или веб-интерфейсом.